# Assignment-14.5

## 2303A51272

## Batch-23

## Task 1 – Image Gallery Website

Create an image gallery web page.

Requirements:

• Display images in a grid layout.

• Use AI to:

o Optimize layout using CSS Grid.

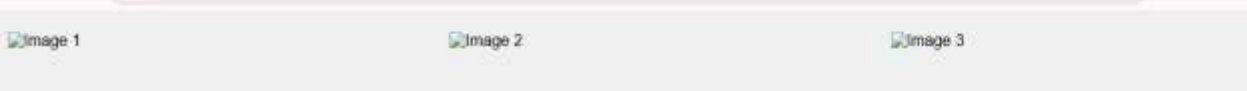
o Add hover zoom or overlay effects.

o Ensure mobile responsiveness.

webpage.html > webpage.html > html > div.gallery > div.gallery-item > img

```html
<!-- create an image gallery web page and display images in grid layout. Optimize layout using CSS Grid and Add hover zoom or overlay effects.Ensure mobile responsiveness.-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Image Gallery</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-color: #f0f0f0;
        }
        .gallery {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
            gap: 15px;
            padding: 20px;
        }
        .gallery-item {
            position: relative;
            overflow: hidden;
        }
        .gallery-item img {
            width: 100%;
            height: auto;
            transition: transform 0.3s ease;
        }
        .gallery-item:hover img {
            transform: scale(1.1);
        }
        .overlay {
            position: absolute;
            top: 0;
            left: 0;
            width: 100%;
            height: 100%;
            background-color: rgba(0, 0, 0, 0.5);
            color: white;
            display: flex;
            align-items: center;
            justify-content: center;
            opacity: 0;
            transition: opacity 0.3s ease;
        }
```

```
            opacity: 0;
            transition: opacity 0.3s ease;
        }
        .gallery-item:hover .overlay {
            opacity: 1;
        }
    </style>
</head>
<body>
</body>
    <div class="gallery">
        <div class="gallery-item">
            <img src="image1.jpg" alt="Image 1">
            <div class="overlay">Image 1 Description</div>
        </div>
        <div class="gallery-item">
            <img src="image2.jpg" alt="Image 2">
            <div class="overlay">Image 2 Description</div>
        </div>
        <div class="gallery-item">
            <img src="image3.jpg" alt="Image 3">
            <div class="overlay">Image 3 Description</div>
        </div>
        <!-- Add more gallery items as needed -->
    </div>
</html>
```




# Task 2 – Profile Card Generator

Create a profile card web page.

Requirements:

• Display user image, name, role, and social media links.

• Use AI to:

o Suggest modern UI colors and fonts.

o Design the card using Flexbox.

o Add hover animation and shadow effects

webpage.html > webpage.html > html > head > style > profile-image

```html
<!-- Create a profile card web page. Display user image, name, role, and social media links,Suggest modern UI colors and fonts, and Design the card using Flexbox. and Add hover animation and shadow e
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title
    <style
        bo
```

This attribute defines the name of a piece of document-level metadata. It should not be set if one of the attributes `itemprop`, `http-equiv` or `charset` is also set.

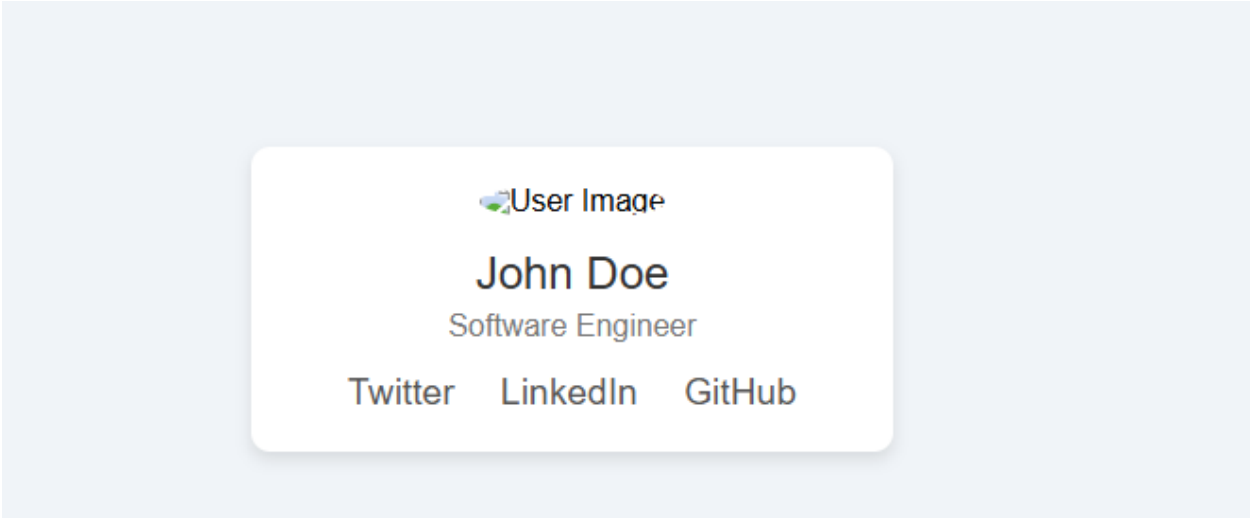
This metadata name is associated with the value contained by the `content` attribute. The possible values for the name attribute are:

- `application-name` which defines the name of the application running in the web page.

  **Note:**
  - Browsers may use this to identify the application. It is different from the `<title>` element, which usually contain the application name, but may also contain information like the document name or a status.
  - Simple web pages shouldn't define an application-name.
- `author` which defines the name of the document's author.
- `description` which contains a short and accurate summary of the content of the page. Several browsers, like Firefox and Opera, use this as the default description of bookmarked pages.
- `generator` which contains the identifier of the software that generated the page.
- `keywords` which contains words relevant to the page's content separated by commas.
- `referrer` which controls the `Referer` HTTP header attached to requests sent from the document.

```html
        width: 300px;
        text-align: center;
        padding: 20px;
        transition: transform 0.3s, box-shadow 0.3s;
    }
    .profile-card:hover {
        transform: translateY(-10px);
        box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
    }
    .profile-image {
        width: 100px;
        height: 100px;
        border-radius: 50%;
        object-fit: cover;
        margin-bottom: 15px;
    }
    .profile-name {
        font-size: 1.5em;
        color: #333333;
        margin-bottom: 5px;
```

```html
    .profile-image {
        border-radius: 50%;
        object-fit: cover;
        margin-bottom: 15px;
    }
    .profile-name {
        font-size: 1.5em;
        color: #333333;
        margin-bottom: 5px;
    }
    .profile-role {
        font-size: 1em;
        color: #777777;
        margin-bottom: 15px;
    }
    .social-links a {
        text-decoration: none;
        color: #555555;
        margin: 0 10px;
        font-size: 1.2em;
    }
    .social-links a:hover {
        color: #0073e6;
    }
</style>
</head>
<body>
    <div class="profile-card">
        <img src="user-image.jpg" alt="User Image" class="profile-image">
        <div class="profile-name">John Doe</div>
        <div class="profile-role">Software Engineer</div>
        <div class="social-links">
            <a href="https://twitter.com/johndoe" target="_blank">Twitter</a>
            <a href="https://linkedin.com/in/johndoe" target="_blank">LinkedIn</a>
            <a href="https://github.com/johndoe" target="_blank">GitHub</a>
        </div>
    </div>
</body>
</html>
```




# Task 3 -Develop a dynamic shopping cart system.

Requirements:

• Add, remove, and update product quantities.

• Display real-time total price.

• Use AI to:

o Manage cart state using JavaScript objects.

o Persist cart data in localStorage.

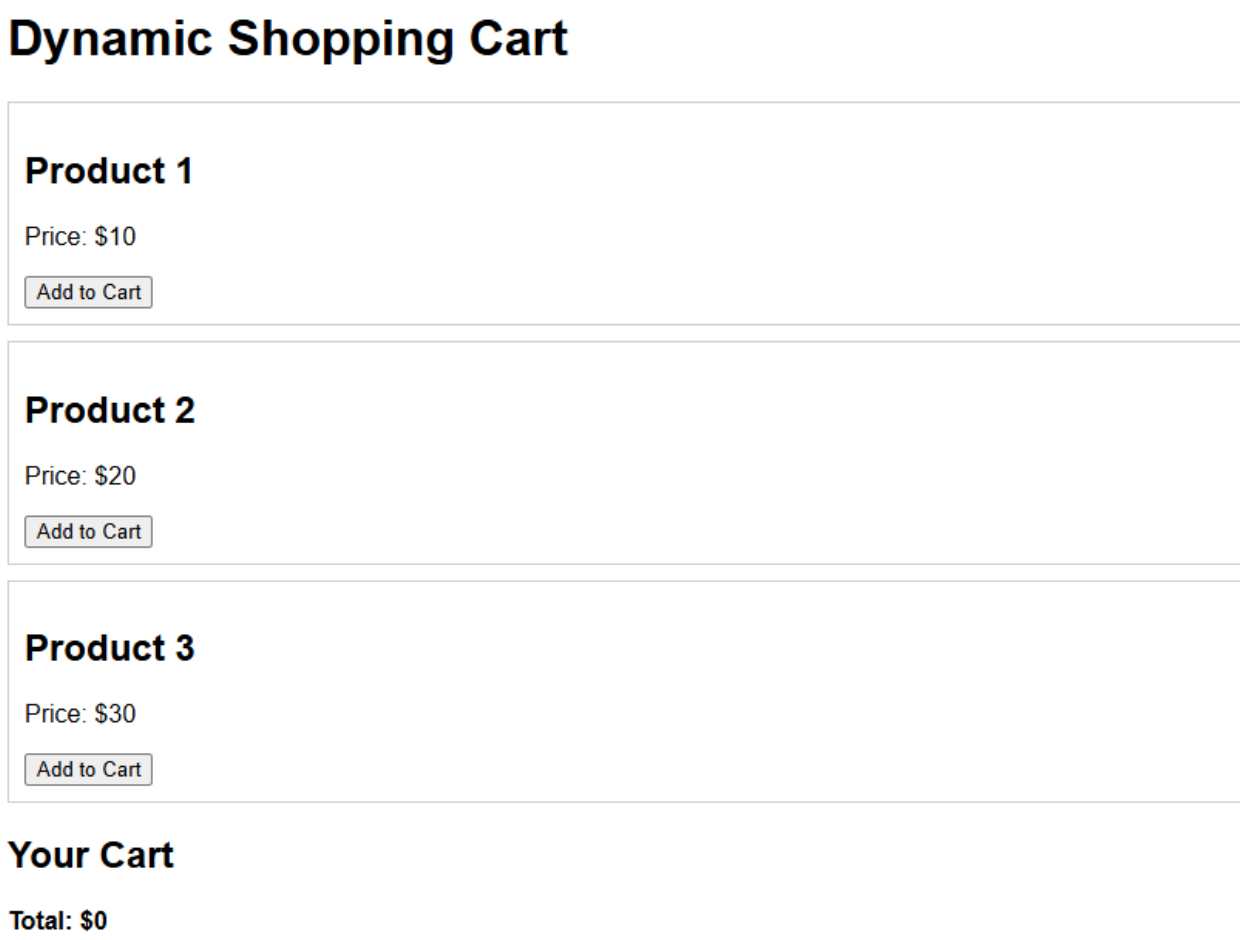
webpage.html > webpage.html > html > body > div#products > div.product

```html
<!--Develop a dynamic shopping cart system.Add, remove, and update product quantities.Display real-time total price.
 and Manage cart state using JavaScript objects.Persist cart data in localStorage.-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dynamic Shopping Cart</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .product {
            border: 1px solid #ccc;
            padding: 10px;
            margin-bottom: 10px;
        }
        .cart {
            margin-top: 20px;
        }
        .cart-item {
            border-bottom: 1px solid #ccc;
            padding: 10px 0;
        }
        .total {
            font-weight: bold;
            margin-top: 10px;
        }
    </style>
</head>
<body>
    <h1>Dynamic Shopping Cart</h1>
    <div id="products">
        <div class="product" data-id="1" data-price="10">
            <h2>Product 1</h2>
            <p>Price: $10</p>
            <button onclick="addToCart(1, 10)">Add to Cart</button>
```

```html
            <p>Price: $10</p>
            <button onclick="addToCart(1, 10)">Add to Cart</button>
        </div>
        <div class="product" data-id="2" data-price="20">
            <h2>Product 2</h2>
            <p>Price: $20</p>
            <button onclick="addToCart(2, 20)">Add to Cart</button>
        </div>
        <div class="product" data-id="3" data-price="30">
            <h2>Product 3</h2>
            <p>Price: $30</p>
            <button onclick="addToCart(3, 30)">Add to Cart</button>
        </div>
    </div>

    <div class="cart">
        <h2>Your Cart</h2>
        <div id="cart-items"></div>
        <div class="total">Total: $<span id="total-price">0</span></div>
    </div>

    <script src="cart.js"></script>
</html>
```




# Task 4 – Role-Based Access Control Application

Create a role-based web application.

Requirements:

• Admin, Editor, User roles.

• Conditional UI rendering.

• Use AI to:

o Design access control logic.

o Maintain secure state transitions.

o Create scalable UI components.

webpage.html > webpage.html > html

```html
<!-- Create a role-based web application. Admin, Editor, User roles.Conditional UI rendering.
and Design access control logic.Maintain secure state transitions.Create scalable UI components.-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Role-Based Web Application</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .admin, .editor, .user {
            display: none;
        }
        .admin {
            background-color: #f8d7da;
            padding: 10px;
            margin-bottom: 10px;
        }
        .editor {
            background-color: #d1ecf1;
            padding: 10px;
            margin-bottom: 10px;
        }
        .user {
            background-color: #d4edda;
            padding: 10px;
            margin-bottom: 10px;
        }
    </style>
</head>
<body>
```

```html
</head>
<body>
    <h1>Welcome to the Role-Based Web Application</h1>
    <div id="adminSection" class="admin">
        <h2>Admin Section</h2>
        <p>Only admins can see this section.</p>
    </div>
    <div id="editorSection" class="editor">
        <h2>Editor Section</h2>
        <p>Only editors can see this section.</p>
    </div>
    <div id="userSection" class="user">
        <h2>User Section</h2>
        <p>Only users can see this section.</p>
    </div>

    <script>
        // Simulate user role (this would typically come from authentication logic)
        const userRole = 'editor'; // Change this to 'admin', 'editor', or 'user'

        // Function to render UI based on user role
        function renderUI(role) {
            if (role === 'admin') {
                document.getElementById('adminSection').style.display = 'block';
            } else if (role === 'editor') {
                document.getElementById('editorSection').style.display = 'block';
            } else if (role === 'user') {
                document.getElementById('userSection').style.display = 'block';
            }
        }

        // Render the UI based on the user's role
        renderUI(userRole);
    </script>
</html>
```

## Welcome to the Role-Based Web Application

### Editor Section

Only editors can see this section.

# Task 5 – Multi-Language Website

Design a multilingual website.

Requirements:

• Switch languages dynamically.

• Persist language preference.

• Use AI to:

o Structure translation files.

o Handle RTL/LTR layouts.

o Improve accessibility.

webpage.html > webpage.html > html

```html
<!-- Design a multilingual website.Switch languages dynamically.Persist language preference. and
Structure translation files. Handle RTL/LTR layouts. Improve accessibility.-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Multilingual Website</title>
    <link rel="stylesheet" href="styles.css">
</head><body>
    <header>
        <nav>
            <ul>
                <li><a href="#" data-lang="en">English</a></li>
                <li><a href="#" data-lang="es">Español</a></li>
                <li><a href="#" data-lang="ar">العربية</a></li>
            </ul>
        </nav>
    </header>
    <main>
        <h1 data-translate="welcome">Welcome to our multilingual website!</h1>
        <p data-translate="description">This website supports multiple languages. You can switch languages dynamically and your preference will be saved.
    </main>
    <script src="script.js"></script>

</html>
```

- English
- Español
- العربية

# Welcome to our multilingual website!

This website supports multiple languages. You can switch languages dynamically and your preference will be saved.

# Task 6 – Online Examination System

Create an online exam interface.

Requirements:

• Timer, questions, and submission.

• Prevent multiple submissions.

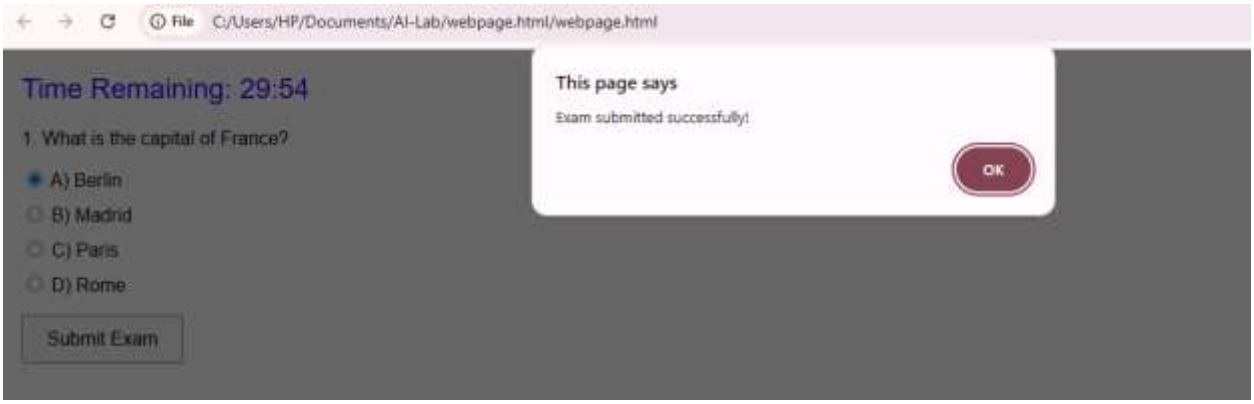
• Use AI to:

o Manage exam state.

o Improve exam UX.

o Ensure accessibility.

webpage.html > webpage.html > ...

```html
<!-- Create an online exam interface.Timer, questions, and submission.Prevent multiple submissions.
 and Manage exam state. Improve exam UX. Ensure accessibilit-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Online Exam Interface</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .timer {
            font-size: 24px;
            color: rgb(64, 0, 255);
            margin-bottom: 20px;
        }
        .question {
            margin-bottom: 15px;
        }
        .options {
            list-style-type: none;
            padding: 0;
        }
        .options li {
            margin-bottom: 10px;
        }
        .submit-btn {
            padding: 10px 20px;
            font-size: 16px;
        }
    </style>
```

```html
    </style>
</head>
<body>
    <div class="timer" id="timer">Time Remaining: 30:00</div>

    <div class="question" id="question1">
        <p>1. What is the capital of France?</p>
        <ul class="options">
            <li><input type="radio" name="q1" value="a"> A) Berlin</li>
            <li><input type="radio" name="q1" value="b"> B) Madrid</li>
            <li><input type="radio" name="q1" value="c"> C) Paris</li>
            <li><input type="radio" name="q1" value="d"> D) Rome</li>
        </ul>
    </div>
    <button class="submit-btn" onclick="submitExam()">Submit Exam</button>
</body>
</html>
<script>
    let timeRemaining = 30 * 60; // 30 minutes in seconds
    const timerElement = document.getElementById('timer');
    const submitButton = document.querySelector('.submit-btn');
    let examSubmitted = false;

    function updateTimer() {
        if (timeRemaining > 0) {
            timeRemaining--;
            const minutes = Math.floor(timeRemaining / 60);
            const seconds = timeRemaining % 60;
            timerElement.textContent = `Time Remaining: ${minutes}:${seconds < 10 ? '0' : ''}${seconds}`;
        } else {
            submitExam();
        }
    }

    function submitExam() {
        if (examSubmitted) {
            alert("You have already submitted the exam.");
            return;
        }
        examSubmitted = true;
        clearInterval(timerInterval);
        alert("Exam submitted successfully!");
        // Here you can add code to handle the submission, e.g., send data to the server
    }

    const timerInterval = setInterval(updateTimer, 1000);
</script
```